RES-AI: Evidence-Bound Readiness for AI Software Engineering Experiments on Shared HPC Infrastructure

Majid Babaei[1*]

[1*]School of Computing, University of the Fraser Valley, BC, Canada.

Corresponding author(s). E-mail(s): majid.babaei@ufv.ca;

Abstract

Researchers across software engineering increasingly conduct experiments in or for AI systems. Executing them on shared clusters introduces a gap between a successful small test and justified admission of a larger job. Configuration checks alone cannot establish that evidence still describes the submitted workload, that required probes ran, or that a successful process produced valid outputs. We present RES-AI, a research prototype separating the scientific specification from its execution plan and evaluating a bounded readiness contract before production submission. The contract binds evidence to the current plan, requires coverage, freshness, memory headroom and repeated observations, and charges failed attempts against the validation budget. A restricted Model Context Protocol interface exposes assessment and refinement proposals to agents; existing Slurm templates provide execution and auditing. An executed synthetic mutation study comprises 1040 policy decisions over 12 fault families and clean controls. The full policy blocks all 240 faulty variants and accepts all 20 clean variants in this deliberately constructed suite, whereas a static validator blocks 40 and a smoke proxy 60. A leave-one-conjunct-out ablation shows that each of the five contract components detects faults no other check detects. Threshold sweeps, 1400 simulated refinement episodes and latency measurements show that decisions follow declared parameters, that charged validation stays within its cap without altering the scientific specification, and that assessment costs milliseconds. They also expose two specification gaps. These results establish implementation behavior under specified inputs, not field failure rates or resource savings. We also provide a real-GPU integration pilot and a prospective paired evaluation protocol for twelve publicly available agent research artifacts. The cross-artifact campaign, production-scale reliability and compute savings remain unestablished.

Keywords: empirical software engineering, AI agents, shared HPC, Slurm, experimental validation, reproducibility, Model Context Protocol

1 Introduction

An increasing share of software engineering researchers are becoming researchers in or for AI systems. The speed of AI development and its growing role in software construction make experimental execution part of the research method itself. Evaluations of code generation, repository repair and software development agents execute generated programs, construct environments, and interact with models over many tasks and seeds. SWE-bench and SWE-agent illustrate this move toward repository-level execution, while rigorous code-generation evaluation requires more than superficial output checks [1–3]. Scientific conclusions therefore depend on both the approach under investigation and the infrastructure that actually runs it.

Shared high-performance computing (HPC) makes substantial experiments accessible without requiring each research group to own an accelerator fleet. It introduces constraints absent from a workstation: allocations expire, resources are heterogeneous, software stacks differ across nodes, and admission follows site policy. A malformed configuration can fail quickly; an input-dependent memory fault or broken checkpoint can remain hidden much longer. A process can also exit successfully while writing incomplete data. Repeated submission without a clear validation contract consumes resources and delays both the researcher and other cluster users.

Reproducibility initiatives emphasize executable artifacts and explicit experimental context [4, 5]. For reviewers, a practical question is whether a result can be regenerated from declared inputs. For an organization considering adoption, it is whether the artifact operates within its resource and deployment constraints. Neither question is answered by a successful scheduler exit code alone. Scientific benchmarking also requires clear measurement conditions and treatment of variation [6]. Resource use matters because validation itself has a cost: a preflight consuming more compute than it saves is not automatically beneficial.

Three gaps motivate this work. First, static validity and execution validity differ: a valid job graph can fail to initialize a model or satisfy an output contract. Second, evidence is conditional: a successful smoke run can become irrelevant after a model, input, software or resource change. Third, validation coverage and cost need to be represented together in the admission decision. An agent repeatedly modifying a workflow can amplify these gaps unless changes invalidate corresponding evidence and retries remain bounded.

We address these gaps with RES-AI, an evidence-bound readiness layer above a Slurm execution engine. A scientific specification identifies the intended experiment; an execution plan identifies how it runs. Smoke probes produce observations against declared obligations. The readiness gate accepts only fresh, matching, sufficiently complete evidence within a budget. A permitted refinement creates a new plan identity and requires new evidence. A small Model Context Protocol (MCP) interface lets agents inspect eligibility without unrestricted submission privileges.

The contribution is narrower than a new scheduler or a reproducibility guarantee. We reuse the job-graph, runner and audit templates supplied by `use-hpc` and identify their pinned revision. We do not claim priority escalation: a short, accurate time request can improve backfill eligibility, but the scheduler controls admission [7]. A small probe also cannot establish that a many-node, multi-day experiment will succeed.

We make four contributions: (1) a readiness contract over scientific identity, execution identity, coverage, resource observations and validation cost; (2) a Python prototype with conservative assessment, bounded refinement proposals, an operator-facing submission wrapper and restricted MCP tools; (3) executed controller studies (a mutation study, a component ablation, a threshold sensitivity analysis, a bounded-refinement simulation and an overhead measurement) and a Slurm GPU integration pilot, all regenerated from archived records; and (4) a twelve-artifact evaluation protocol separating infrastructure reliability, scientific preservation and total resource cost. The fourth contribution is a protocol, not a completed generalizability result.

2 Background and Running Example

2.1 Shared resources and empirical AI studies

Slurm manages resource requests, allocation and execution [8]. A partition defines an administrative resource set and its constraints; accounts and quality-of-service policies determine what a user may request. Dependencies express job precedence, and arrays represent repeated experiment cells. Backfill can start a lower-priority request when doing so does not delay a higher-priority job's expected start. It depends on requested resources and time limits rather than an application independently acquiring priority [7].

For an AI experiment, allocation is only one part of the context. Weights, tokenizer, data revision, input lengths, precision, prompts, test harness and seeds can affect both demand and outcomes. Serving systems such as vLLM explicitly manage inference memory [9]; an orchestration layer should not infer numerical reproducibility from available memory. A valid deployment is a prerequisite for evaluating an approach, not evidence of its effectiveness.

Workflow systems already automate dependencies and execution across resources [10–12], containers improve portability [13], and context capture supports computational reuse [14, 15]. Our question is consequently not whether jobs can be orchestrated, but when validation evidence justifies dispatch of a particular production specification.

2.2 An illustrative running example

Consider a team evaluating a multi-agent code-generation framework across 64 GPUs. Its pipeline loads models, distributes tasks, executes generated programs and aggregates results. A long input could exceed memory, a worker could fail to join a collective, or aggregation could drop unfinished tasks. A fault appearing on the third day would be expensive, but the day and scale in this example are hypothetical, not measurements from our study.

A short single-GPU test exercises imports and loading, but not necessarily maximum input size, checkpoint recovery or inter-node communication. It provides evidence for its exercised obligations, not readiness for the complete 64-GPU plan. RES-AI makes the missing obligations explicit. The team must add representative boundary-input, checkpoint and distributed probes, or narrow its claim. No fixed fifteen-minute turnaround is promised: the queue and required test scale determine the actual time.

Suppose a boundary-input probe approaches the memory limit. Reducing an inference microbatch can be operational if the experiment permits it and preserves intended outcomes. Reducing effective training batch size can change the experiment. The implemented proposal therefore requires a scientific declaration of batch invariance, which must separately be tested. An out-of-memory event never authorizes changing the model, data or evaluation metric.

2.3 Identity, topology and evidence

Definitions are needed because a pass/fail flag without scope is ambiguous. We first distinguish intended science from execution configuration, then identify the evidence connecting them.

Definition 1 (Experiment identity) Let S be the scientific specification, X the execution plan, O the mandatory probe set, B the validation policy and budget, and I declared input identities. The plan identity is $h = H(S, X, O, B, I)$, using canonical SHA-256. S records scientific choices such as model, data, prompts, seeds and scoring; X records resource shape and operational settings; I binds declared files to full content hashes.

Changing X invalidates old-plan evidence; changing S additionally changes the scientific experiment. Hash equality establishes identity of declared content, not authenticity, exhaustive dependency capture or deterministic execution.

Topology can inform feasible request shapes and probe selection, but scheduler descriptions need not measure physical network performance. Unknown attributes must remain explicit.

Definition 2 (Observed resource graph) A resource snapshot is $G_t = (V_t, E_t, A_t)$, with timestamp t, advertised nodes or accelerators V_t, topology relations E_t and attributes A_t. Edge bandwidth and latency may be unknown. Scheduler-reported relations and measurements inside allocations are distinct evidence sources.

Discovery archives what the scheduler reports; it neither reconstructs measured link weights nor optimizes placement. In the example, sharing an advertised switch does not replace a distributed probe at the required allocation shape.

Each attempt must identify what it exercised and observed. Otherwise successful execution can be mistaken for comprehensive validation.

Definition 3 (Smoke evidence) An attempt record e_i contains a distinct identifier, plan hash h_i, timestamp t_i, process status, passed obligations C_i, output validity, peak memory fraction m_i and allocated-instance process time c_i. Bundle E retains all supplied attempts, including failed and obsolete attempts.

Obligations can include imports, model initialization, a boundary input, output schema/count, checkpoint round trip and distributed collective completion. Their implementation is workload-specific; names alone do not establish adequate testing. The CUDA pilot implements the first five for a small synthetic network and contains no distributed probe.

Readiness is a conjunction so an obligatory failure cannot be hidden by an average score. With the two most recent matching attempts e_a, e_b, lifetime T, memory limit $m_{\max}$, stability tolerance ϵ, attempt cap K and compute cap C, the rule is

$$\begin{aligned} \mathrm{Ready}(P,E,t) = \; & \mathrm{Valid}(P) \wedge |E| \le K \wedge \sum_{e_i \in E} c_i \le C \\ & \wedge \bigwedge_{i \in \{a,b\}} [h_i = h \wedge \mathrm{OK}(e_i) \wedge O \subseteq C_i \wedge \mathrm{OutputValid}(e_i)] \\ & \wedge \bigwedge_{i \in \{a,b\}} [0 \le t - t_i \le T \wedge 0 \le m_i \le m_{\max}] \wedge \Delta_{\mathrm{smoke}} \le \epsilon, \end{aligned} \tag{1}$$

where $P = (S, X, O, B, I)$ and $\Delta_{\mathrm{smoke}} = |m_a - m_b|$. Records are ordered by timestamp for selection. All attempt identifiers must be nonempty and unique; every c_i must be positive and finite. Missing evidence blocks readiness. Distinct executions do not establish statistical independence. Memory difference is a stability diagnostic, not a convergence theorem for stochastic agents.

3 Approach

3.1 Overview

Figure 1 organizes RES-AI into three tiers. An access tier separates an AI agent, which reaches the system only through restricted MCP tools, from the operator, who executes probes, reviews proposals and requests dispatch. The middle tier is the RES-AI readiness layer, whose six numbered components connect an experiment plan to an admission decision. The bottom tier is the inherited execution substrate: the pinned `use-hpc` templates and the Slurm scheduler of the shared cluster. RES-AI adds the readiness layer and reuses the substrate unchanged.

The experiment plan (1) separates scientific choices S from execution settings X and records the mandatory probes O, validation policy B and input identities I; its hash h identifies the exact plan. Resource discovery (2) issues read-only scheduler queries and archives a snapshot G_t that informs the reviewed choice of X. The scheduler retains control of allocation and admission.

Bounded smoke probes (3) exercise the declared obligations through the `use-hpc` runner under explicit, time-limited allocations started by the operator. Each attempt

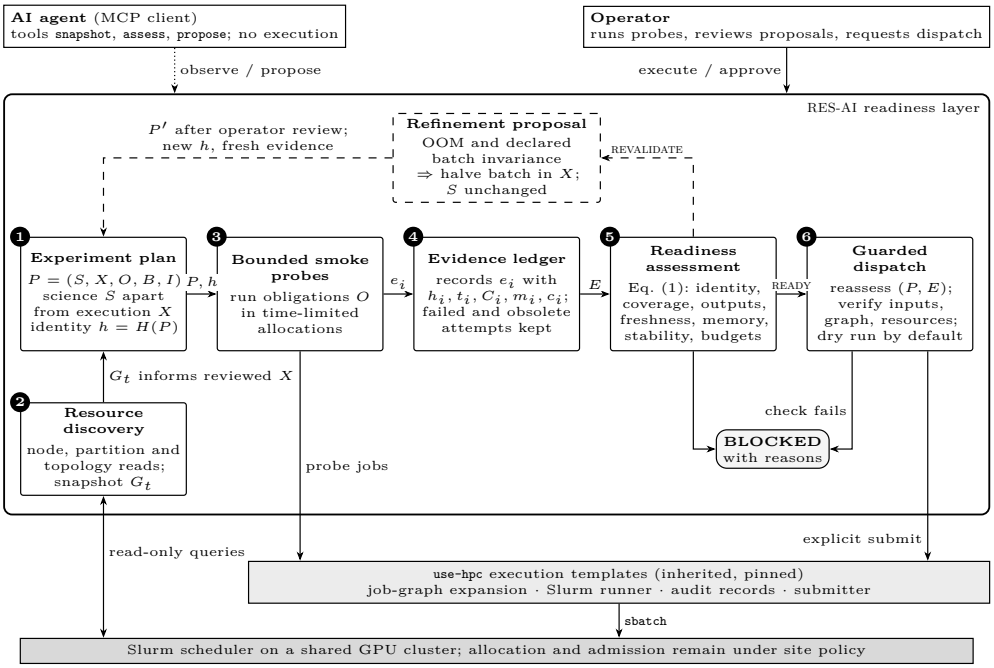


Figure 1 Layered architecture of RES-AI. Numbered components form the readiness layer; shaded tiers are inherited and unchanged. Solid arrows carry data or decisions: the plan P and its identity h, attempt records e_i, accumulated evidence E and the resource snapshot G_t. Dashed elements form the only revision path, which requires operator review, yields a new identity and invalidates earlier evidence for admission. The dotted agent connection permits observation and proposals but neither execution nor submission. OOM denotes out of memory.

yields a record e_i in the evidence ledger (4), carrying its plan identity, timestamp, passed obligations, output validity, peak memory fraction and cost. Failed and obsolete attempts remain in the ledger for validation-cost accounting. Readiness assessment (5) evaluates Equation (1) over the current plan and accumulated evidence E: identities must match, the two most recent observations must be fresh, complete, valid and stable, and attempt and compute budgets must hold. Missing or invalid evidence yields BLOCKED with reasons.

A READY decision passes to guarded dispatch (6), which reassesses readiness immediately before dispatch and verifies declared files, the resolved execution graph and the supported resource mapping. Any failed check yields BLOCKED. Otherwise the wrapper produces a dry run by default and invokes the upstream submitter only on explicit operator request.

The dashed path applies only when a permitted refinement exists. After a matching out-of-memory failure, assessment returns REVALIDATE with a proposal that halves the batch size in X, provided the scientific contract declares batch invariance; S is never altered. The operator reviews the proposal and adopts the revised plan P', whose new identity prevents reuse of earlier evidence, and collects fresh matching observations. Without an applicable proposal, the operator must diagnose the failure,

obtain missing evidence or stop. Agents may call discovery, assessment and proposal tools, whereas probe execution and submission remain operator actions. Algorithms 1–3 detail these interactions; unattended repair and resubmission are not implemented.

Algorithm 1 DiscoverClusterTopology: archive resource observations

Require: Scheduler read interfaces
Ensure: Timestamped snapshot G_t with explicit unknowns
$R \leftarrow \emptyset$; $t \leftarrow$ CurrentTime
for q in {node inventory, partitions, topology} **do**
    $R[q] \leftarrow$ ReadWithTimeout(q) ▷ output, status or unavailable
return $(R, t,$ bandwidth unknown, latency unknown$)$

Algorithm 2 ExecuteSmokeTestLoop: assess a bounded iteration

Require: Plan P, accumulated evidence E, snapshot G_t
Ensure: READY, REVALIDATE with proposal, or BLOCKED with reasons
CheckRequestFeasibility(P, G_t) ▷ operator checks site constraints
$r \leftarrow$ AssessReadiness(P, E) ▷ Equation 1
if r.ready **then**
    return READY, r
if attempt or compute budget exhausted **then**
    return BLOCKED, r
$P' \leftarrow$ ProposeRefinement(P, E)
if $P' \neq \perp$ **then**
    return REVALIDATE, P' ▷ collect new matching evidence
return BLOCKED, r ▷ diagnose fault or obtain missing probes

Algorithm 3 OrchestrateFullExperiment: operator-facing dispatch

Require: Plan P, evidence E, dispatch mode d (dry-run by default)
$r \leftarrow$ AssessReadiness(P, E)
r.reasons $\leftarrow r$.reasons $\cup$ VerifyDeclaredInputs(P)
r.reasons $\leftarrow r$.reasons $\cup$ ValidateExecutionGraph(P)
PrintDecision(r)
if r.reasons $\neq \emptyset$ **then**
    return BLOCKED
return UpstreamSubmit(d) ▷ explicit operator submission

3.2 Resource discovery and request selection

Algorithm 1 performs light metadata reads. Each read is bounded in time and preserves failures, preventing unavailable topology from silently becoming an apparently healthy empty cluster. Discovery archives what the operator can inspect; it does not place jobs.

Request selection respects authorized accounts, partitions, accelerator type, memory and walltime. Let $\mathcal{P}(j, t)$ denote the scheduler's priority for job j at time t. It is observed or queried where available; it is not a value that RES-AI can independently increase. Selection resides in reviewed configuration. Short requests may create backfill opportunities, but their effect on waiting time must be measured. Discovery and monitoring must respect site limits on scheduler queries.

3.3 Coverage-directed validation

An obligation list explains why a probe is needed before execution. In the running example, imports and initialization test deployment, a boundary input tests a declared envelope, an output check tests schema/cardinality, a checkpoint probe tests recovery, and a collective probe tests communication. A test cannot satisfy an unexercised obligation by completing quickly. A single-GPU observation must not be relabeled as evidence for a distributed workload.

Algorithm 2 uses the two most recent observations for the exact plan. Older or different-plan records do not establish readiness but retain their compute charges. A recent failed observation blocks admission even if an earlier attempt passed. Two observations reduce dependence on one accidental success; they do not characterize rare faults. Evidence expiry is a policy choice, not an empirically discovered boundary of resource stability.

The lifetime, memory envelope and validation budget are predeclared policy parameters rather than universal constants. Their experimental values appear in Section 5.3. A production study must justify these choices for its workload and distinguish framework-allocated memory from all memory used on a device.

3.4 Bounded refinement and scientific preservation

A feedback mechanism needs permitted actions and stopping rules. The approach permits one action: halve a batch size greater than one after an out-of-memory classification, only if the scientific contract declares batch invariance and the failing evidence matches the plan. The action modifies the execution subdocument and never the scientific one, and it is returned as a proposal for operator review rather than executed.

The revised hash prevents old records from validating the new configuration. Two new matching observations are needed, with previous costs retained. Unknown faults produce no proposal. Attempt and compute caps bound assessment eligibility, and scheduler limits bound the actual allocation. An autonomous wall-clock budget controller is outside the scope of the approach.

Preserving JSON fields is necessary but insufficient for preserving the scientific estimand. Training batch changes can alter optimization; inference scheduling and numerical nondeterminism can alter outputs. The prospective campaign therefore

tests permitted repairs against predeclared tolerances. A model substitution, precision change or revised dataset creates a new scientific specification rather than an invisible recovery action.

3.5 Guarded execution and agent access

The execution backend expands the experiment graph, submits dependencies, runs probes and retains provenance. These mechanisms are inherited from `use-hpc`; the additional contribution is the admission contract connecting a particular plan to evidence and a bounded refinement proposal.

Algorithm 3 reassesses readiness immediately before dispatch and checks whether declared inputs and the resolved execution graph still match the assessed plan. An unsupported mapping or changed input blocks the request. A successful check produces a dry run unless the operator explicitly requests submission. Scientific metadata declares intended behavior; it does not prove the scientific semantics of arbitrary commands. The gate is a workflow convention, not a scheduler security boundary.

Agents access discovery, assessment and refinement proposals through a restricted MCP interface, while an operator controls evidence collection and dispatch. This separation makes the decision and its reasons inspectable without granting an agent unrestricted execution capability. Section 5.2 describes how each component is realized, including modules, protocol version, supported mappings and implementation limits.

4 Related Work

4.1 Workflow execution and provenance

Pegasus, Snakemake, Nextflow and Parsl address dependencies, resource execution and portability [10–12, 16]. CWL standardizes reuse; Sumatra and Sacred capture experimental context [14, 15, 17]. An auditable workflow and environment capture are therefore foundations, not novel RES-AI features. A container improves mobility but does not itself establish probe coverage or freshness after a resource change.

Our contribution is a particular admission contract above execution machinery: mandatory evidence must still apply to the exact specification, and its acquisition cost must remain bounded. Such a policy can complement established workflow systems; this artifact integrates only the supplied Slurm template. We do not claim other systems cannot implement equivalent policies.

4.2 Agents that execute and repair workflows

SWE-agent and OpenHands use agent-computer interfaces for substantial software tasks [2, 18]. ExecutionAgent executes tests of arbitrary projects, making iterative environment construction a relevant comparator [19]. MLAgentBench evaluates agents on machine-learning experimentation [20]. These studies make feedback-driven execution central and show why static-only comparison cannot isolate the value of RES-AI.

Setup recovery differs from readiness for an expensive production allocation. Our protocol retains native recovery and checkpoints and includes smoke-then-production

Table 1 Relationship to established mechanisms. Entries describe relevant roles, not exhaustive feature comparisons.

Mechanism	Established role	Additional readiness question
Workflow engine	Dependencies and execution	Does evidence authorize this plan?
Context/container	Identity and portability	Were mandatory probes passed?
Agent setup repair	Iterative environment construction	Are cost and scientific changes bounded?
Slurm/backfill	Policy-controlled allocation	Is the request feasible and justified?
`use-hpc` templates	Smoke runs and audit	Is evidence current, complete and stable?

as a strong baseline. MCP standardizes tools but does not establish their observations' validity. The contribution is exposure of a bounded assessment contract, not a claim that a standardized interface makes experiments reliable.

4.3 Empirical rigor and artifact evaluation

Reproducibility work emphasizes methods and execution context; empirical standards make study design and evidence explicit [4, 21]. EMSE case-study guidance emphasizes defined cases, collection procedures and threats [22]. Scientific benchmarking requires measurement variability and execution conditions to be considered [6]. Together, these motivate separating controller conformance, integration feasibility and field effectiveness.

5 Evaluation Design

5.1 Research questions and evidence levels

We evaluate RES-AI with four research questions.

- **RQ1 (admission):** Does evidence-bound admission block invalid episodes before production without blocking valid ones?
- **RQ2 (cost):** What does validation cost, and is that cost bounded once failed and recovery attempts are charged?
- **RQ3 (preservation):** Do permitted operational changes leave the intended scientific experiment unchanged?
- **RQ4 (components):** What does each conjunct of Equation (1) contribute beyond a successful smoke run?

The evidence comes from three levels of increasing realism and decreasing completeness. Level A consists of executed controller studies on synthetic records. These are a mutation study, a leave-one-conjunct-out ablation, a threshold sensitivity analysis, a bounded-refinement simulation and an assessment-overhead measurement. Level B is a real-GPU integration pilot with small synthetic tensors on the Nibi cluster. Level C

Table 2 Research questions, endpoints and evidence. "Executed" analyses produced the results in Section 6; "prospective" analyses belong to the campaign protocol in Section 5.4 and have not been run.

Question	Endpoint	Executed evidence (Levels A, B)	Prospective evidence (Level C)
RQ1	Detection, false blocks, production escapes	Mutation study; threshold sensitivity; four CUDA probe modes	Natural and injected faults across twelve artifacts
RQ2	Allocated cost, time to valid completion	Slurm accounting of the pilot; refinement cost bounds; assessment latency	Paired failure-penalized cost against smoke-then-production
RQ3	Scientific identity, output equivalence	Invariance of S across all refinement proposals	Equivalence tests within predeclared margins
RQ4	Contribution of each conjunct	Leave-one-conjunct-out ablation on the mutation suite	Paired artifact-level ablations

is a prospective paired campaign over twelve research artifacts; its availability checks are complete, while installation, adapter validation and comparative reruns are not. Table 2 maps each question to its endpoints and to the evidence currently available. No Level A or B observation is presented as a result of the twelve-artifact campaign.

5.2 Implementation details

This subsection collects the implementation decisions that realize the approach of Section 3. Table 3 lists the modules, their origin and their size.

Prototype and execution backend.

RES-AI version 0.1.0 is a Python package that requires Python 3.11 or newer. The policy, the command-line interface and the MCP server use only the standard library, which keeps the trusted assessment path free of third-party dependencies. The execution backend reuses the `use-hpc` templates pinned at revision `f9dc7d6`. Matrix expansion, dependencies, step execution, completion markers, manifests, audit ledgers and result collection are inherited. Source snapshots and full revisions accompany the artifact.

Identity and readiness.

`resai.core` computes h as SHA-256 over canonical JSON (sorted keys, compact separators, NaN and infinity rejected), so semantically equal plans hash identically regardless of key order. `validate` rejects malformed plans before any evidence is read. These include missing fields, an empty or duplicated probe list, nonpositive budgets, an attempt cap below two, and policy values outside their domains ($0 \leq m_{\max} \leq 1$,

Table 3 Prototype and study modules. LOC counts nonblank source lines in the archived revision. "Inherited" modules come unchanged from `use-hpc` at revision `f9dc7d6`; "Study" modules generate the reported evidence.

Module	Origin	LOC	Responsibility
`core.py`	New	130	Canonical identity, plan validation, readiness (Eq. 1), refinement, discovery
`pipeline.py`	New	99	Input binding, graph/resource correspondence, dry-run or explicit dispatch
`mcp.py`	New	52	Stdio JSON-RPC server exposing three read/propose tools
`cli.py`	New	21	`discover`, `gate` and `refine` commands
`hpclib.py`	Inherited	180	Matrix expansion, dependency resolution and static graph validation
`run_stage.py`	Inherited	184	Step execution, completion markers and manifests
`audit.py`	Inherited	287	Audit ledger and job watcher
`collect_results.py`	Inherited	282	Result collection across array cells
`run_local.py`	Study	100	Level A mutation generator and comparator policies
`run_analyses.py`	Study	176	Ablation, sensitivity, refinement and overhead analyses
`probe_gpu.py`	Study	70	Level B CUDA probes with five obligations

$0 \leq \epsilon \leq 1$, $T \geq 1\,\mathrm{s}$). `readiness` then evaluates every conjunct of Equation (1) and returns a sorted set of reason codes rather than stopping at the first failure. An operator therefore sees all violated terms at once. Records with nonnumeric or nonfinite telemetry are rejected immediately with an `invalid_telemetry` reason. Table 6 lists the reason codes that the conjuncts emit. The prototype trusts operator-supplied records: hashes, timestamps and costs are not tamper-resistant attestations, and the accounting is not a trusted service.

Discovery and refinement.

Discovery (Algorithm 1) issues `sinfo -json`, `scontrol show partition` and `scontrol show topology`, each with a 20-second timeout. It archives return codes and raw output, records an unavailable command as unavailable rather than as an empty cluster, and fixes bandwidth and latency to "unknown". It returns raw scheduler observations rather than a normalized node–edge database and implements no topology-aware placement. Metadata reads have per-read timeouts but no global polling limiter; the inherited audit watcher waits at least sixty seconds between updates. `refine` implements the single permitted action. It halves a batch size greater than one only when the most recent record matches the current plan, is classified as out of memory, the scientific contract declares batch invariance, and both attempt and compute budgets remain. It deep-copies the plan, modifies only the execution subdocument and returns the proposal without executing it.

Dispatch.

Algorithm 3 is realized in `resai.pipeline`. Declared hashes must bind the selected pipeline's `experiments.json` and `cluster.env` as well as all other declared inputs; the wrapper checks declared local files but does not discover transitive dependencies. It verifies the resolved graph identity, the scientific metadata declaration, the batch variable and the allocation shape, and rejects raw scheduler overrides in GPU stages. The current mapping supports homogeneous GPU stages with explicit batch configuration and HH:MM:SS walltimes; unsupported shapes block dispatch. Site values containing carriage returns or newlines are rejected. The default is an upstream dry run, and `-submit` explicitly invokes submission. Scientific metadata remains an operator assertion about command semantics. Direct upstream invocation bypasses the wrapper, and concurrent changes after verification and duplicate submission remain limitations.

Agent interface.

The stdio MCP server exposes `cluster_snapshot`, `assess_readiness` and `propose_refinement`. It supports initialization, tool listing and tool calls using newline-delimited JSON-RPC for the 2024-11-05 protocol [23]. File arguments resolve inside the workspace after symlink resolution, and external paths are rejected. No shell or submission tool is exposed. The command-line interface offers the same three operations (`discover`, `gate`, `refine`); `gate` exits with status 2 when evidence is not ready, so shell pipelines can stop on a blocked decision. This is a single-user prototype without interoperability certification or a security audit.

Research-artifact adapter.

The MapCoder adapter pins upstream revision `c87e6d91` and archives its patch and provenance. A HumanEval-only import patch removes inactive competitive-programming dependencies. A local Transformers backend replaces the cloud backend, while the selected strategy's prompts and traversal are retained. Generated programs run in an AST-restricted subprocess with CPU, address-space and wall-time limits, rather than the original thread evaluator, and assertions remain enabled. This narrow evaluator is not a hardened sandbox. Per-attempt directories prevent old output files from appearing fresh. The resume probe reloads persisted results into fresh strategy objects and checks that no additional inference is needed. Requirements pin PyTorch 2.14.1, Transformers 4.57.6 and tiktoken 0.12.0.

Analysis pipeline and tests.

CSV and JSON records retain attempt identifiers, source hashes, software and device observations, and Slurm accounting. Each generator asserts the expected record cardinality before producing a table or plot; for example, the ablation's full variant must reproduce the mutation study's RES-AI column exactly. All figures are vector PDFs generated from the archived records. Thirteen unit tests cover policy edge cases, path confinement and graph/resource mismatches, and a claims checker verifies that reported counts, citations and frozen identities match the raw data.

Table 4 Workloads and execution settings. MapCoder entries describe the frozen requested configuration; its execution status is reported separately. CPU and RAM values are per allocation.

Factor	Controller studies (A)	CUDA integration (B)	Staged MapCoder
Workload	12 fault families + clean control; parameter sweeps; refinement episodes	MLP 512–1024–64; input 128×512	HumanEval/0; Qwen2.5-Coder-7B-Instruct
Repetitions	Seeds 0–19 per cell; 200 timing repeats	Two cells; four modes each	Two fresh sequential model processes
Accelerator	None	One H100 1g.10gb MIG per cell	One full H100 requested
Host resources	Local workstation	2 CPU cores; 12 GB RAM	4 CPU cores; 48 GB RAM
Allocation ceiling	No Slurm allocation	15 min per cell; 30 instance-min total	15 min for both processes
Output oracle	Injected property / clean control	Probe coverage and output validity	Record validity; task correctness separate

5.3 Experimental setup

Environments.

Level A analyses ran on a Windows 11 workstation (Intel x86-64) with Python 3.13.14. Level B records report PyTorch 2.14.1 and CUDA runtime 13.2 on an H100 1g.10gb MIG instance of Nibi. CUDA execution and environment preparation take place inside scheduled allocations. MIG-instance seconds and full-H100 seconds are different resource units and are never pooled. Table 4 summarizes the workloads.

Policy parameters.

Table 5 lists the readiness parameters, fixed before analysis. They exercise the contract rather than representing optimized thresholds; the sensitivity analysis below varies them explicitly. A production study must justify its own envelope and should distinguish framework-allocated memory from all memory used on a device.

Mutation study (RQ1).

A generator creates a valid plan with two successful records, varying a data-identity token and a baseline memory fraction drawn uniformly from $[0.4, 0.7]$ under 20 deterministic seeds. For each seed, one of 12 operators corrupts a single property, and a clean control is retained. Table 6 defines every operator, the term of Equation (1) it violates and the reason codes it produces. This yields 260 cases per policy and 1040 decisions overall. Four policies are compared. *Direct* admission accepts everything and serves as a reference floor. *Static* applies the actual upstream graph validator. The *smoke proxy* combines that validator with successful process statuses and output-file existence. It models what a passing smoke run shows, but it does not execute the

Table 5 Readiness parameters fixed before analysis. Process-instance budgets govern the supplied evidence; Slurm ceilings separately bound allocations.

Parameter	Symbol	Controller / CUDA	MapCoder
Matching observations required	—	2	2
Evidence lifetime (s)	T	3,600	3,600
Maximum memory fraction	$m_{\max}$	0.90	0.90
Maximum between-attempt difference	ϵ	0.05	0.05
Maximum supplied attempts	K	4	4
Process-instance validation budget (s)	C	1,200	1,800

entire upstream smoke pipeline or every check an experienced user would perform. Finally, RES-AI combines upstream validation with Equation (1). The oracle follows the operator: a clean fixture should pass and an intentionally violated property should block.

Ablation (RQ4).

Because Equation (1) is a conjunction, removing a conjunct is equivalent to ignoring the reason codes it emits. We group reason codes into five components: *Identity* ($h_i = h$, distinct identifiers), *Outcome* (OK, coverage, output validity), *Freshness* ($t - t_i \leq T$), *Memory* (headroom and stability) and *Budget* (attempt cap, compute cap and valid accounting). A variant admits a case when the upstream validator passes and every emitted reason belongs to its removed component. All 5 variants are evaluated on the mutation suite, holding the fixtures, seeds and validator fixed. The ablation measures the logical contribution of each conjunct on constructed faults, not the frequency of the corresponding faults in practice.

Threshold sensitivity (RQ1).

To test whether decisions follow the declared parameters, we sweep one evidence property at a time across three settings of the parameter that governs it, holding all else valid. The swept properties are the age of both matching records ($T \in \{1800, 3600, 7200\}$ s, 250 s steps), a footprint scale applied to each seed's baseline memory ($m_{\max} \in \{0.8, 0.9, 0.95\}$), the between-attempt memory difference ($\epsilon \in \{0.02, 0.05, 0.1\}$) and the total charged validation time ($C \in \{600, 1200, 1800\}$ s). Each point is assessed for 20 seeds, giving 9180 assessments over 12 parameter settings. Plans are rehashed after every change so that only the swept property differs.

Bounded refinement (RQ2, RQ3).

We simulate Algorithm 2 with an operator who adopts every proposal. Peak memory scales linearly with batch size from each seed's baseline at batch 8. A probe whose peak exceeds device capacity fails with an out-of-memory classification, and every probe is charged 60 s. Episodes start from batch sizes $\{8, 16, 32, 64, 128\}$ under attempt caps $K \in \{2, \ldots, 8\}$, with and without declared batch invariance, for 20 seeds each: 1400 episodes. An episode ends READY, BLOCKED with no applicable proposal,

Table 6 Fault operators of the mutation study. Each operator corrupts one property of an otherwise valid plan and evidence pair. Component is the ablation group that owns the violated term (Graph faults are caught by the upstream validator). First blocker is the weakest policy, in the order static, smoke proxy, RES-AI, that blocks all 20 variants; each stronger policy blocks them too, and direct admission blocks none.

Fault family	Mutation	Violated term	Component	Emitted reason codes	First blocker
GPU request absent	Remove the stage GPU request	Valid(P)	Graph	— (graph validator)	Static
Dependency invalid	Depend on a nonexistent stage	Valid(P)	Graph	— (graph validator)	Static
Process failed	Set last status to failed	OK(e_i)	Outcome	`execution_failure`	Smoke proxy
Required probe absent	Drop the checkpoint obligation	$O \subseteq C_i$	Outcome	`coverage`	RES-AI
Output invalid	Mark the present output invalid	OutputValid(e_i)	Outcome	`output_contract`	RES-AI
Plan identity changed	Change the scientific model	$h_i = h$	Identity	`two_distinct_attempts`	RES-AI
Attempt ID duplicated	Reuse the first attempt ID	distinct IDs	Identity	`duplicate_or_missing_attempt_id`, `two_distinct_attempts`	RES-AI
Evidence expired	Move the last record to $t = 1$	$t - t_i \leq T$	Freshness	`stale_evidence`	RES-AI
Memory limit exceeded	Set peak memory to 0.98 (OOM)	$m_i \leq m_{\max}$	Memory	`memory_headroom`, `unstable_memory`	RES-AI
Memory unstable	Add 0.12 to the last peak	$\Delta_{\text{smoke}} \leq \epsilon$	Memory	`unstable_memory`	RES-AI
Validation budget exceeded	Charge 1,300 s to the last record	$\sum c_i \leq C$	Budget	`compute_budget`	RES-AI
Accounting absent	Delete the last cost c_i	finite $c_i > 0$	Budget	`invalid_telemetry:gpu_seconds`	RES-AI

or BLOCKED because the attempt or compute budget is exhausted. The scientific subdocument is hashed before and after every proposal. The memory model is a deliberate simplification that exercises the control loop; it does not predict the memory behavior of real workloads.

Assessment overhead (RQ2).

We time a single `readiness` call for evidence ledgers of 2^1 to 2^{11} matching records, 200 repetitions each, using a monotonic high-resolution clock. We report the median and interquartile range and compare them with the shortest measured Level B probe allocation.

Integration pilot (RQ1, RQ2).

The Level B pilot submits a two-cell Slurm array through `use-hpc`, followed by CPU analysis and collection under a CPU association. Each cell runs four modes. Clean mode checks imports, initialization, input execution, output cardinality and checkpoint round trip. The other modes remove an output row, omit checkpoint testing,

or raise a controlled runtime exception after CUDA execution. Failures are recorded without terminating the enclosing collection stage, so a process-level failure cannot silently remove an inconvenient observation. Analysis requires exactly four modes by two repeats, pairs repeats within each mode and applies Equation (1). A per-mode plan identity is assigned retrospectively to describe the known probe; this is not pre-execution admission of the full graph. Slurm allocation and queue times are reported separately from timed probe computation, which excludes installation, imports and scheduler overhead. This is synthetic integration, not an agent-paper rerun or a large-model benchmark.

Staged MapCoder pilot (RQ3).

Before expanding to twelve artifacts, we apply the approach to the local-model MapCoder configuration [24]. The specification fixes a cached Qwen revision (`c03e6d35`), greedy decoding with seed zero, three recalled examples, five debugging opportunities and one outer attempt. Input and generation caps are 12,288 and 4,096 tokens, batch size is one, and batch invariance is false. Task, model revision, prompts and scoring belong to S. Consequently, the local-model adapter is a new deployment configuration, not a reproduction of published aggregate accuracy. The protocol calls for two fresh model processes executed sequentially in a scheduled interactive smoke allocation. Obligations cover imports, model initialization, task execution, output schema and cardinality, and persisted-result resume. Scientific task correctness is recorded separately: a valid record of an unsuccessful generated program is not a malformed output. We require an initial blocked assessment without evidence, then assess collected observations; a dependent CPU job collects evidence and prints a dispatch dry run. No larger campaign is dispatched before author review.

Analysis of executed levels.

Level A cases are deterministic functions of their seeds, and seed variants are not independent research artifacts. We therefore report counts, fractions and per-family matrices without significance tests or population confidence intervals. Operators were authored with knowledge of the policy, so a perfect count establishes conformance to the specification, not accuracy on naturally occurring faults. Level B reports all eight raw records and the corresponding Slurm accounting.

5.4 Prospective twelve-artifact campaign

Hypotheses.

The campaign tests four primary hypotheses at the artifact level; all are two-sided except the equivalence hypothesis. H_{01} states zero paired difference in pre-production detection rate between full RES-AI and the strongest smoke comparator, with false-block rates reported alongside. H_{02} states zero paired difference in the predeclared failure-penalized time/resource endpoint. For a declared output difference d and equivalence margin δ, $H_{03} : d \leq -\delta \ \vee \ d \geq \delta$ is tested using two one-sided tests against $-\delta < d < \delta$; equivalence must be established, not inferred from a nonsignificant difference. H_{04} states zero paired difference between the full policy and each predeclared

Table 7 Candidate artifacts of the prospective campaign. Every comparative execution status is currently *not run*. HEADs are availability observations, not validated paper-version pins.

Artifact	Intended workflow	Observed HEAD
SWE-agent [2]	repository repair	`3ea751c0`
OpenHands [18]	software development	`11eba891`
AutoCodeRover [25]	structure-aware repair	`585d3e63`
MetaGPT [26]	role-based development	`11cdf466`
ChatDev [27]	communicative development	`a2ea4a26`
AgentCoder [28]	generation and testing	`d8538d63`
MapCoder [24]	competitive programming	`c87e6d91`
RepairAgent [29]	Java repair	`5d9dc8de`
ExecutionAgent [19]	test environment setup	`8fb16de8`
CodeAct [30]	executable code actions	`d607f56c`
HyperAgent [31]	generalist SE	`c3092f26`
AgileCoder [32]	sprint-based development	`f9924185`

ablation on its targeted outcome. None of these hypotheses is tested with the synthetic seed variants of Level A.

Subjects.

We assembled twelve real agent research artifacts spanning repository repair, generation, collaborative development and test-environment construction (Table 7). Paper links and full revisions are in `experiments/subjects.json`. Availability was checked through the public repository API on 23 October 2026; a current revision is an availability observation, not automatically the paper's evaluated version.

The inventory is purposive, not a systematic literature-review funnel. Before claiming generality, the campaign must archive searches of ICSE, FSE, ASE, ISSTA, EMSE, TSE and relevant AI venues for 2023–2026, combining "agent", "software engineering", "code generation", "program repair" and "test execution". Screening must retain identifiers, duplicate decisions, queries, paper and repository links, and exclusions. Eligibility requires accessible paper-compatible code, permission for the experiment, a reduced executable workload, an output oracle and deployment compatible with site policy. Infeasible candidates remain in the ledger, and replacements must be recorded before comparative outcomes are inspected.

The set is concentrated in generation and repair. CodeAct includes tasks beyond software engineering, so its inclusion requires restricting evaluation to software-oriented tasks. Cloud-API and local-model workloads must be separated: API experiments can establish deployment and CPU/allocation observations but cannot establish local GPU savings. A local-endpoint port is a new configuration requiring capability checks and pinned weights. Container-dependent artifacts require a site-approved alternative rather than privileged Docker on compute nodes.

Procedure.

For each eligible artifact, we freeze paper-compatible source, environment, benchmark subset, model, serving configuration, prompts, token limits, seeds and scoring, and specify the resource shape and permitted repairs before running any condition. A held-out validation subset is used to design probes; final benchmark outcomes are not fed into the admission policy. Native retries, recovery and checkpoints remain enabled in every condition to avoid handicapping comparators. The conditions are direct native submission, upstream static validation, upstream smoke-then-production, full RES-AI, and three ablations removing coverage obligations, identity/freshness checks and refinement proposals. All conditions share the artifact adapter. The design crosses twelve artifacts, seven conditions, five paired seeds and four episode classes (clean, environment fault, resource fault and output-contract fault): 1,680 capped episodes if all artifacts are eligible. This is a design count, not submitted work or a cost estimate, and a feasibility pilot must fix per-artifact caps and the total budget before launch. Conditions are randomized within artifact, site/resource and submission-time blocks. Caches are held constant or varied as a declared factor, and documented faults are injected at the same point in paired conditions. Natural faults are retained separately because their frequency and detectability may differ from seeded faults. Human intervention, transcript identity, repair actions, completion, validated output and allocation identifiers are logged. Failed installations and time-capped episodes remain in the denominator.

Metrics.

RQ1 measures detection among faulty episodes and false blocking among valid clean episodes, alongside production escapes and blocking reasons. Rejecting everything yields high detection and unusable availability, so both outcomes are mandatory. RQ2 charges all validation, recovery and production allocations:

$$C_{\text{total}} = \sum_{j \in \text{validation,recovery,production}} a_j \, \text{ElapsedAllocated}(j), \tag{2}$$

where a_j counts allocated accelerator instances. CPU-seconds, instance type, API token cost and queue time are reported separately, and unlike accelerators are never pooled into unlabeled GPU-hours. Early rejection may avoid wasted work while preventing a valid result, so completion rate and time-to-result are needed to interpret resource use. Each episode receives a predeclared cap, and actual allocated consumption is always retained. A separate failure-penalized endpoint assigns the cap to unsuccessful episodes for the primary intention-to-run comparison; this imputation is not labeled measured consumption. A sensitivity analysis uses restricted mean time to valid completion under explicit censoring rules. Queue delay is submission-to-start, not allocated GPU time. RQ3 requires exact immutable-input identity and artifact-specific output tolerances fixed before comparison: deterministic outputs may require equality, whereas stochastic task success needs equivalence margins and sufficient repetitions. RQ4 compares the full policy against each ablation on targeted outcomes,

Table 8 Mutation study outcomes per policy. Admitted faults are invalid fixture records accepted by a policy, not observed production failures. False blocks use 20 clean controls; seed variants are not independent artifacts.

Policy	Blocked faults	Admitted faults	Detection (%)	False blocks
Direct	0/240	240/240	0.0	0/20
Static	40/240	200/240	16.7	0/20
Smoke proxy	60/240	180/240	25.0	0/20
RES-AI	240/240	0/240	100.0	0/20

including extra validation cost, and effects must persist against the full upstream smoke condition, not merely direct submission.

Statistical analysis.

The primary unit is the artifact; seeds, tasks and injected faults are nested observations. We compute artifact-level paired differences, report their distribution with a bootstrap interval that resamples artifacts, and perform a paired randomization test over artifact-level signs. With twelve eligible artifacts, all 2^{12} sign configurations can be enumerated. Holm correction is applied within the predeclared primary family, and effect sizes are reported with their uncertainty. Twelve artifacts provide limited precision, so a prospective precision or power assessment is required before fixing the campaign size [33].

6 Results

We report results by research question. Each subsection ends with a boxed answer that states what the executed evidence supports and what remains open.

6.1 RQ1: Admission decisions

Mutation study.

Table 8 and Figure 2a derive from the archived `decisions.csv`. Direct admission blocks no injected fault. Static validation blocks the 40 variants of the two graph-fault families (16.7%), and the smoke proxy additionally blocks the 20 process-failure variants (25.0%). Full readiness blocks all 240 faulty variants and admits all 20 clean controls. No policy produces a false block, so this suite exhibits no detection/availability trade-off within its narrow valid envelope.

The per-family view (Table 6) explains the aggregate difference. Records with incomplete probe coverage, invalid but present outputs, a changed plan identity, expired evidence, memory violations or invalid accounting all pass both the static validator and the smoke proxy, because each of these faults leaves the job graph valid and the process status successful. A malformed but present output illustrates why file presence is useful for dependencies but insufficient for schema and cardinality. Successful

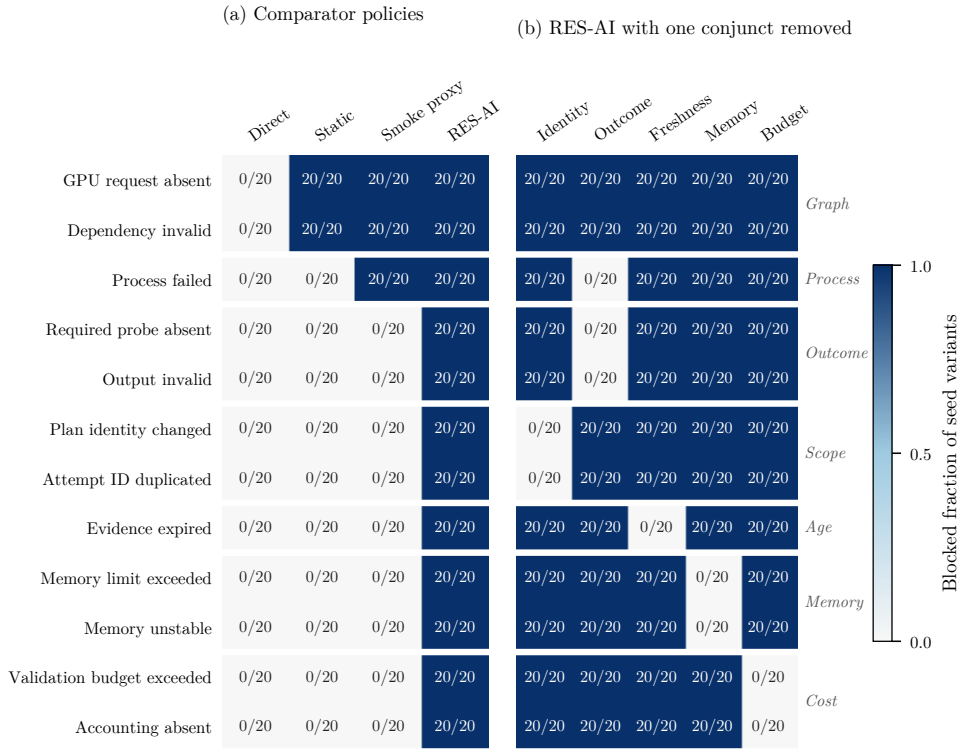


Figure 2 Blocked variants per fault family, out of 20 seeds. (a) Comparator policies of the mutation study. (b) Full RES-AI with one conjunct of Equation (1) removed (RQ4). Italic labels group families by the contract concern they violate. Each removed conjunct reopens exactly the families it alone detects.

observations of an obsolete plan remain useful history without validating a changed plan.

Threshold sensitivity.

Figure 3 shows that every decision boundary moves with its declared parameter and with nothing else. Admission drops from 20/20 to 0/20 between the last grid point at or below T and the first above it (panel a), and between charged totals of 1,200 s and 1,250 s when $C = 1200$ s (panel d). For memory headroom (panel b), admission decreases gradually because each seed has its own baseline footprint. At a footprint scale of 1.8, 2, 8 and 9 of 20 seeds remain admissible for $m_{\max} = 0.8$, 0.9 and 0.95. The static validator and the smoke proxy admit every point of every sweep (dotted lines), because none of the swept properties is visible to them. One implementation detail surfaced at the stability boundary (panel c). When $|m_a - m_b|$ equals ϵ exactly, 7 of 20 seeds pass for $\epsilon = 0.05$ and 15 of 20 for $\epsilon = 0.1$, because floating-point rounding of the difference decides the tie. Exact-boundary decisions are therefore not reproducible

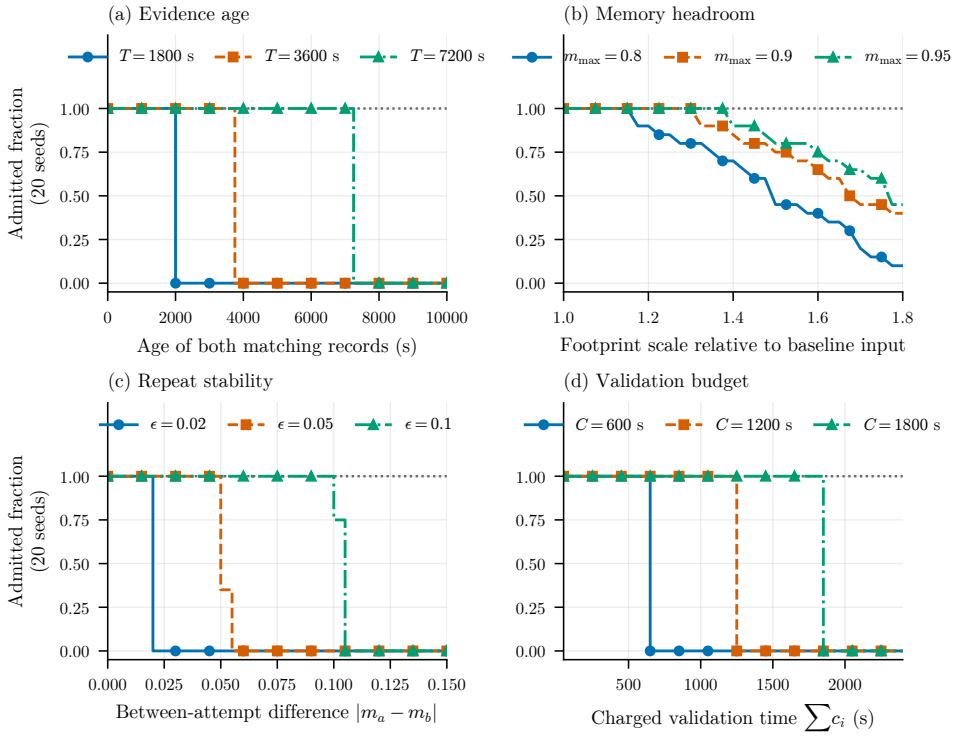


Figure 3 Threshold sensitivity of RES-AI admission. Each panel sweeps one evidence property while all other properties remain valid; lines correspond to three settings of the governing parameter and each point aggregates 20 seeds. The dotted line at 1.0 is shared by the static validator and the smoke proxy, which never respond to these properties. Step lines hold the value until the next grid point.

across inputs, and a production deployment should either quantize telemetry or state a tolerance.

GPU integration.

All eight CUDA probe records were collected from the two completed GPU array cells (job `24888989`), and the dependent analysis job `24888990` completed. The clean pair passed the retrospective readiness assessment; corrupted-output, missing-checkpoint and runtime-failure pairs were blocked. Slurm reports 81 allocated MIG-instance seconds across the two cells, with 159 seconds of queue delay for each. The timed probe computations sum to 3.723 instance-seconds; the difference includes environment setup, imports and job overhead. These are measurements of this small integration workload, not saved production compute or evidence about an agent's task accuracy. Table 9 shows that the pilot reproduces two faults that an exit-code check would miss. The corrupt-output processes complete successfully but fail coverage and output validity, and the missing-checkpoint processes produce valid outputs but fail a mandatory

Table 9 CUDA diagnostics for two repetitions on Nibi. t_0, t_1 are timed probe-process seconds. Probes gives the passed obligations out of five in *each* repetition; Output gives valid records out of two. Assessment uses the pair.

Mode	t_0 (s)	t_1 (s)	Probes	Output	Assessment / reasons
Clean	0.599	0.585	5/5	2/2	Ready
Corrupt output	0.619	0.334	4/5	0/2	Blocked: coverage; output validity
No checkpoint	0.458	0.440	4/5	2/2	Blocked: coverage
Runtime failure	0.366	0.322	3/5	0/2	Blocked: process failure; coverage; output validity

obligation. The deliberately raised runtime failure is visible both to status checks and to the readiness rule. Memory headroom is ample in this small workload, so the pilot provides no test of large-model memory pressure.

Answer to RQ1

On constructed faults, evidence-bound admission blocked all 240 faulty variants without false blocks, whereas the smoke proxy blocked 25.0%. Decision boundaries follow the declared parameters exactly, except for floating-point ties at the stability threshold. On real hardware, RES-AI blocked two fault modes that exit statuses did not reveal. These results establish conformance to the specification, not detection rates for natural faults.

6.2 RQ2: Validation cost and its bounds

Measured allocation cost.

Figure 4 separates waiting, allocation and probe computation in the Level B pilot. The two GPU cells start after the same 159-second queue delay and consume 39 and 42 allocated MIG-instance seconds. Their timed probe computations sum to 3.723 seconds across all eight records, so timed computation accounts for under 5% of the allocated cost. Reporting only the inner computation would understate the cost of this integration check roughly twentyfold. The remainder includes setup, imports and execution overhead; the archive does not support a causal decomposition of it.

Bounded refinement cost.

Figure 5 and Table 10 report the 1400 simulated refinement episodes. Charged validation never exceeded the attempt-derived ceiling of $60K$ s in any episode (Figure 5b). Mean cost grows with K only until the episodes terminate, then stays flat: additional budget is not consumed once a decision is reached. The cap also limits reachable repairs. With the study setting $K = 4$, an initial batch of 64 reached READY in only 2 of 20 episodes, and a batch of 128 in none, because two matching observations plus every failed out-of-memory attempt must fit within K. With $K = 8$, every initial batch from 16 to 128 converged to READY in the same 14 of 20 episodes, at a mean of 156–336 s of charged validation. Without declared batch invariance, no proposal was ever produced; episodes that failed with out-of-memory stopped after a single 60-second probe.

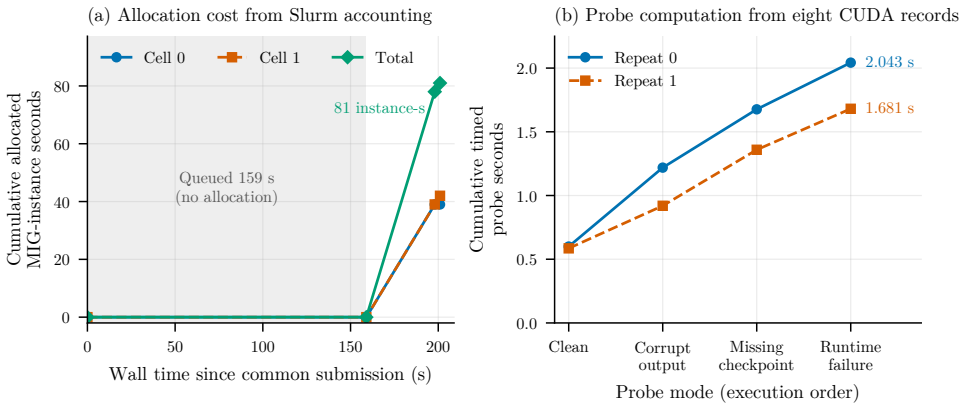


Figure 4 Cost accounting of the Nibi integration pilot. (a) Cumulative allocated MIG-instance seconds reconstructed from Slurm start and end times; queued time accrues no allocated cost. (b) Cumulative timed probe computation in execution order, including the deliberately failing mode. The panels use different accounting boundaries and are not utilization or savings estimates.

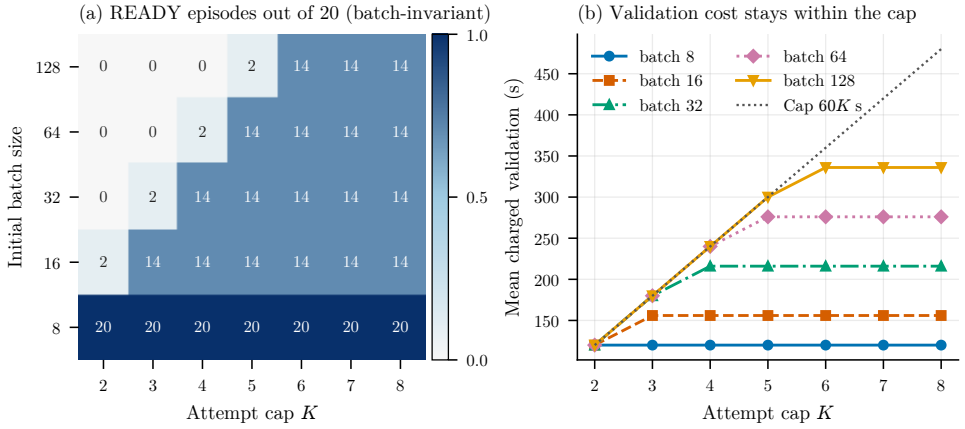


Figure 5 Bounded refinement under the attempt cap K (simulated, batch-invariant contract). (a) Episodes reaching READY out of 20 seeds by initial batch size and K. (b) Mean charged validation time, which never exceeds the $60K$ s ceiling (dotted) and stops growing once episodes terminate.

Assessment overhead.

Figure 6 shows that a readiness assessment costs a median of 0.022 ms for a two-record ledger and 6.0 ms for 2048 records, growing approximately linearly with ledger size. Even at 2048 records, an assessment costs less than 2×10^{-4} of the shortest measured probe allocation (39 s). The cost of the gate itself is therefore negligible; the cost of validation lies in the probe allocations it requires.

Table 10 Terminal decisions of simulated refinement episodes, out of 20 seeds per row. Ready: READY reached; No prop.: blocked because the contract permitted no further proposal; Cap hit: blocked because the attempt or compute cap was reached; Cost: mean charged validation, with every probe charged 60 s.

	$K = 4$ (study setting)				$K = 8$			
Initial batch	Ready	No prop.	Cap hit	Cost (s)	Ready	No prop.	Cap hit	Cost (s)
Batch-invariant contract								
8	20	0	0	120	20	0	0	120
16	14	6	0	156	14	6	0	156
32	14	6	0	216	14	6	0	216
64	2	0	18	240	14	6	0	276
128	0	0	20	240	14	6	0	336
Batch invariance not declared								
8	20	0	0	120	20	0	0	120
16	2	18	0	84	2	18	0	84
32	0	20	0	60	0	20	0	60
64	0	20	0	60	0	20	0	60
128	0	20	0	60	0	20	0	60

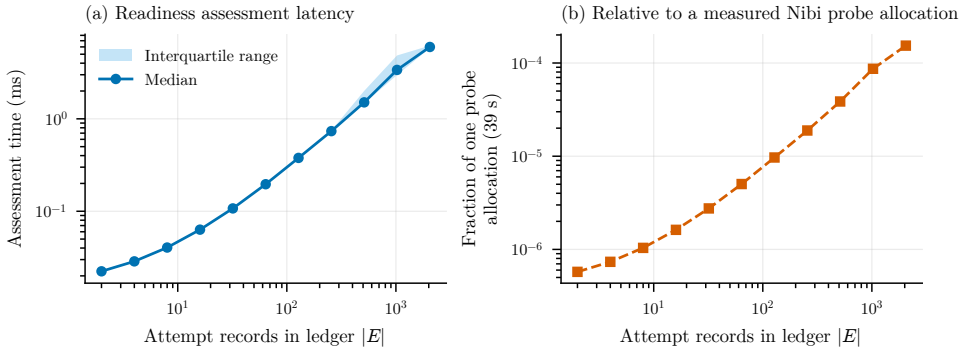


Figure 6 Assessment overhead on the analysis workstation. (a) Median and interquartile range of 200 timed calls per ledger size (log–log). (b) The same medians as a fraction of the shortest measured Nibi probe allocation.

Answer to RQ2

Validation cost is dominated by allocations, not by assessment: the pilot's timed computation was under 5% of its 81 allocated instance-seconds, and the gate costs at most milliseconds. The attempt and compute caps strictly bound charged validation and also limit how many refinements can complete. Whether validation saves resources relative to smoke-then-production remains a campaign question.

6.3 RQ3: Scientific preservation

Across the 1400 episodes, RES-AI produced 1024 refinement proposals. The hash of the scientific subdocument S was identical before and after each of them (0 changes). All proposals came from batch-invariant contracts, and none from contracts without the declaration. This confirms structural preservation: the implementation never rewrites model, data, prompts, seeds or scoring. Structural preservation is necessary but not sufficient for preserving the scientific estimand. Training batch changes can alter optimization, and inference scheduling and numerical nondeterminism can alter outputs.

The initial MapCoder arrays were canceled while pending: first to repair carriage-return configuration defects, then to use the site's documented interactive debugging route after batch admission had no estimated start. A MIG interactive request was also canceled after resource inspection found drained nodes and occupied eligible slices. At the archived status check, full-H100 interactive allocation `24890766` was queued for two sequential smoke executions under a separate plan identity. No task-success or readiness result is asserted before collection. Semantic preservation therefore remains untested by executed evidence; the campaign tests it with equivalence margins fixed before comparison.

Answer to RQ3

No refinement proposal altered the scientific specification, and proposals required an explicit batch-invariance declaration. Whether permitted operational changes preserve experimental outcomes, rather than declared fields, has not yet been measured.

6.4 RQ4: Contribution of contract components

Table 11 and Figure 2b show that every conjunct is necessary on this suite. Removing any single component reopens exactly the fault families that only it detects, lowering detection to between 75.0% (Outcome) and 91.7% (Freshness). No component is redundant: no family is detected by two different components alone, apart from the graph faults that the upstream validator catches under every variant. Removing a component never introduced false blocks. The Outcome component contributes most because it covers three families, but one of those, process failure, is also caught by the smoke proxy. The components that a successful smoke run cannot replace are therefore identity, freshness, memory and budget, together with coverage and output validity.

The refinement study exposes a gap that the ablation cannot. In 6 of 20 seeds, halving the batch produced a peak footprint between 0.9 and 1.0 of device capacity (Table 10, column "No prop."). This violates $m_{\max}$ but does not fail with out-of-memory. The Memory conjunct correctly blocks these plans. However, the single implemented refinement fires only on an out-of-memory classification, so it proposes nothing, and the episode stops even though one further halving would satisfy the contract. The memory conjunct and the refinement rule thus use different triggers. Aligning them is a direct improvement for future versions.

Table 11 Leave-one-conjunct-out ablation on the mutation suite (240 faulty and 20 clean cases per variant). No variant blocked a clean control.

Variant	Removed terms of Eq. (1)	Blocked	Det. (%)	Fault families that escape
Full RES-AI	none	240/240	100.0	none
−Identity	$h_i = h$; distinct IDs	200/240	83.3	Plan identity changed; Attempt ID duplicated
−Outcome	OK; $O \subseteq C_i$; output	180/240	75.0	Process failed; Required probe absent; Output invalid
−Freshness	$t - t_i \leq T$	220/240	91.7	Evidence expired
−Memory	$m_i \leq m_{\max}$; $\Delta_{\text{smoke}} \leq \epsilon$	200/240	83.3	Memory limit exceeded; Memory unstable
−Budget	$\lvert E\rvert \leq K$; $\sum c_i \leq C$; valid c_i	200/240	83.3	Validation budget exceeded; Accounting absent

Answer to RQ4

Each of the five components of Equation (1) detects fault families that no other component and no smoke check detects, so none can be removed without reopening an escape path. The refinement rule is narrower than the memory conjunct and misses headroom violations that do not fail with out-of-memory.

7 Discussion

7.1 Readiness as scoped evidence

Readiness is useful when expressed as evidence with scope. A researcher can explain eligibility, a reviewer can identify tested deployment conditions, and an adopter can determine what must be repeated on a different cluster. Explicit incompleteness is actionable: missing distributed evidence calls for another probe or a narrower claim rather than unqualified extrapolation from a local run. The ablation supports this framing. Every conjunct detected faults that a successful smoke run did not reveal (Section 6.4), so a smoke run's success should be reported with the obligations it exercised and the plan it described.

The executed analyses also show where the specification needs refinement. Ties at the stability threshold depend on floating-point rounding (Section 6.1), and the refinement rule reacts to out-of-memory failures but not to headroom violations that stop short of failure (Section 6.4). Both are implementation-level gaps, found by exercising the contract systematically rather than by inspecting individual runs. This supports making such analyses part of the artifact rather than a one-off check.

7.2 Resource efficiency is comparative

Resource efficiency is a comparative outcome rather than a property conferred by smoke tests. Validation consumes time and can hold scarce devices. For short reliable workloads, direct execution may be cheaper; for long workloads with detectable early faults, bounded validation may help. A simple break-even condition is $C_{\text{validation}} < pC_{\text{avoidable failure}}$, where p is the deployment-specific probability of a failure that the probes actually detect. Neither p nor avoided production cost is estimated by the controller studies. The executed evidence constrains only the left-hand side. Assessment itself is negligible (Section 6.2), allocation overhead dominated the pilot's probe computation, and the attempt cap bounds charged validation at K probe allocations. A team's saved allocation also does not establish cluster-wide utilization gains.

The cap is a policy lever with a visible trade-off. A small K bounds cost tightly but prevents refinements that need several halvings from completing (Table 10); a larger K admits them at proportionally higher validation cost. The choice should follow from the expected cost of a production failure, not from a default.

7.3 Bounded automation must preserve the research question

An agent can make a workflow run by changing models, tasks or numerical settings, potentially invalidating the original experiment. The scientific/execution distinction makes that change reviewable, and the refinement study confirms that the implementation never altered S. Semantic preservation still needs evidence, and batch invariance is particularly workload-dependent. Our refinement is consequently a constrained proposal, not unrestricted self-repair.

7.4 Self-application

An approach for managing HPC experiments itself requires HPC experiments to establish its value. We therefore applied its staged workflow to its own development, progressing from local contract checks to a bounded CUDA integration run and then to one research artifact before considering the twelve-artifact campaign. This makes the proposed discipline concrete: preserve failed or canceled submissions, bind evidence to the configuration actually tested, and require review before expanding the experimental scope. The MapCoder submissions illustrate the point. Three requests were retired before execution, after a configuration defect and unavailable capacity were identified, and each resource-plan change invalidated earlier evidence while the scientific configuration remained fixed. Scheduler admission remains an external constraint; a queued request provides no execution evidence. Self-application can expose implementation and operational defects, but it is not independent validation of effectiveness.

7.5 Transfer

Generality depends on transfer conditions. The candidate artifacts vary in workflow, but several depend on cloud APIs or containers, requirements that MCP does not resolve. The likely transferable mechanism is the contract and evidence schema; probes,

oracles and feasible resource requests remain artifact- and site-specific. Current evidence supports a reproducible prototype and a path to evaluation, not a claim that twelve diverse studies have already benefited.

8 Threats to Validity

Construct validity. Synthetic mutations measure policy behavior on supplied records, not scientific correctness or actual failures prevented. The smoke proxy approximates selected upstream checks. The refinement simulation assumes memory linear in batch size and a fixed per-probe cost, so its convergence counts describe the control loop under that model, not real workloads. Framework-allocated memory excludes some driver and process overhead. The small CUDA network does not approximate long-context inference, agent coordination or training. Retrospective identities in the integration analysis are not pre-execution certificates.

Internal validity. The same developer designed the policy, the operators, the ablation grouping and the oracle, creating favorable-test risk. The ablation removes reason codes after assessment, which is equivalent to removing a conjunct only because every conjunct is evaluated independently. The assessment returns early for nonnumeric telemetry, which the Budget group absorbs. Malformed records may be rejected or raise errors; the prototype has not undergone exhaustive fuzzing. Operator-controlled records can omit costs or assert false coverage. The wrapper is bypassable and has a verification-to-execution race. Inherited completion markers are not content-addressed and can reuse state in ways not fully protected by the wrapper. These limitations prevent security and end-to-end correctness claims.

External validity. One Nibi pilot cannot establish cross-site effects, network behavior or fair-share outcomes. Overhead was timed on one workstation, so absolute latencies will differ elsewhere, although the comparison with probe allocations spans four orders of magnitude. The twelve candidates are purposively selected and concentrated in generation and repair. Current repositories may differ from paper versions, and execution eligibility is pending. API and local-inference workloads have different accounting boundaries. No many-day, multi-node or 64-GPU production experiment is evaluated.

Conclusion validity. Seed variants are not independent studies and receive no inferential tests; deterministic sweeps report exact counts rather than estimates. The planned campaign has a small effective sample and may not establish equivalence. Selecting after outcomes, excluding failed installations or counting successful jobs alone would bias results; the protocol retains such records. Compute savings, long-run reliability and widespread adoption remain hypotheses.

Reproducibility. The execution templates are pinned and attributed. Numbers from the unpublished support manuscript are not reused as new findings. The archive retains controller decisions, ablation, sensitivity, refinement and overhead records, seeds, source hashes, repository-availability records and pilot evidence, and every reported number is regenerated from them. Identity does not capture undeclared dependencies. Release licensing and archival metadata require finalization before public distribution.

9 Conclusion

RES-AI makes readiness an explicit, bounded decision over identity, coverage and observed resource behavior. The prototype adds assessment and constrained refinement proposals above existing Slurm machinery and exposes them through MCP. The executed studies show that the full contract blocks every constructed fault without false blocks, and that each of its five components is necessary. They also show that decisions follow the declared parameters, that charged validation stays within its cap while proposals leave the scientific specification unchanged, and that assessment costs milliseconds. They also exposed two specification gaps: floating-point ties at the stability threshold and a refinement trigger narrower than the memory conjunct. The GPU pilot exercises limited integration on real hardware. The twelve-artifact protocol identifies the evidence still needed to determine whether these mechanisms save resources while preserving scientific outcomes in real AI software engineering studies.

Declarations

Funding. Funding and resource acknowledgements require author confirmation before submission.
Competing interests. The author develops the prototype and reused execution framework; no independent evaluator is claimed.
Human participants. The reported controller and GPU-probe experiments involve no human-subject data.
Data and code availability. Code, observations, analysis and the prospective protocol accompany the RES-AI repository. Execution templates originate from https://github.com/ai4se4ai-lab/use-hpc, pinned in `hpc/UPSTREAM.json`. A permanent artifact DOI has not been assigned.
Use of generative AI. AI coding and writing assistance was used to implement the prototype, draft the manuscript and conduct three methodological, architectural and literature reviews. These reviews do not constitute independent human peer review. The author must verify final claims, citations and journal-specific disclosure before submission.

References

[1] Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: SWE-bench: Can language models resolve real-world GitHub issues? In: Proc. 12th International Conference on Learning Representations (ICLR) (2024)

[2] Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O.: SWE-agent: Agent-computer interfaces enable automated software engineering. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37 (2024)

[3] Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 36 (2023)

[4] Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E., Larochelle, H.: Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). Journal of Machine Learning Research **22**(164), 1–20 (2021)

[5] Association for Computing Machinery: Artifact Review and Badging, Version 1.1. https://www.acm.org/publications/policies/artifact-review-and-badging-current (2020)

[6] Hoefler, T., Belli, R.: Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC), p. 73. ACM, New York, NY, USA (2015). https://doi.org/10.1145/2807591.2807644

[7] SchedMD: Slurm Workload Manager: Scheduling Configuration Guide. https://slurm.schedmd.com/sched_config.html. Accessed 23 October 2026 (2026)

[8] Yoo, A.B., Jette, M.A., Grondona, M.: SLURM: Simple Linux utility for resource management. In: Job Scheduling Strategies for Parallel Processing (JSSPP). LNCS, vol. 2862, pp. 44–60. Springer, Berlin, Heidelberg (2003). https://doi.org/10.1007/10968987_3

[9] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with PagedAttention. In: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), pp. 611–626. ACM, New York, NY, USA (2023)

[10] Deelman, E., Vahi, K., Juve, G., Rynge, M., Callaghan, S., Maechling, P.J., Mayani, R., Chen, W., Silva, R.F., Livny, M., Wenger, K.: Pegasus, a workflow management system for science automation. Future Generation Computer Systems **46**, 17–35 (2015) https://doi.org/10.1016/j.future.2014.10.008

[11] Mölder, F., Jablonski, K.P., Letcher, B., Hall, M.B., Tomkins-Tinch, C.H., Sochat, V., Forster, J., Lee, S., Twardziok, S.O., Kanitz, A., *et al.*: Sustainable data analysis with Snakemake. F1000Research **10**, 33 (2021) https://doi.org/10.12688/f1000research.29032.1

[12] Babuji, Y., Woodard, A., Li, Z., Katz, D.S., Clifford, B., Kumar, R., Lacinski, L., Chard, R., Wozniak, J.M., Foster, I., Wilde, M., Chard, K.: Parsl: Pervasive parallel programming in Python. In: Proc. 28th International Symposium on High-Performance Parallel and Distributed Computing (HPDC), pp. 25–36 (2019). https://doi.org/10.1145/3307681.3325400

[13] Kurtzer, G.M., Sochat, V., Bauer, M.W.: Singularity: Scientific containers for mobility of compute. PLoS ONE **12**(5), 0177459 (2017)

[14] Davison, A.: Automated capture of experiment context for easier reproducibility in computational research. Computing in Science & Engineering **14**(4), 48–56 (2012) https://doi.org/10.1109/MCSE.2012.41

[15] Crusoe, M.R., Abeln, S., Iosup, A., Amstutz, P., Chilton, J., Tijanić, N., Ménager, H., Soiland-Reyes, S., Gavrilovic, B., Goble, C., The CWL Community: Methods included: Standardizing computational reuse and portability with the Common Workflow Language. Communications of the ACM **65**(6), 54–63 (2022) https://doi.org/10.1145/3486897

[16] Di Tommaso, P., Chatzou, M., Floden, E.W., Barja, P.P., Palumbo, E., Notredame, C.: Nextflow enables reproducible computational workflows. Nature Biotechnology **35**(4), 316–319 (2017) https://doi.org/10.1038/nbt.3820

[17] Greff, K., Klein, A., Chovanec, M., Hutter, F., Schmidhuber, J.: The Sacred infrastructure for computational research. In: Proc. 16th Python in Science Conference (SciPy), pp. 49–56 (2017). https://doi.org/10.25080/shinma-7f4c6e7-008

[18] Wang, X., *et al.*: OpenHands: An open platform for AI software developers as generalist agents. In: International Conference on Learning Representations (ICLR) (2025)

[19] Bouzenia, I., Pradel, M.: You name it, I run it: An LLM agent to execute tests of arbitrary projects. In: Proc. ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA) (2025). arXiv:2412.10133

[20] Huang, Q., Vora, J., Liang, P., Leskovec, J.: MLAgentBench: Evaluating language agents on machine learning experimentation. In: Proc. 41st International Conference on Machine Learning (ICML). PMLR, vol. 235 (2024)

[21] Ralph, P., et al.: Empirical Standards for Software Engineering Research. https://arxiv.org/abs/2010.03525. Version 2 (2021)

[22] Runeson, P., Höst, M.: Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering **14**, 131–164 (2009) https://doi.org/10.1007/s10664-008-9102-8

[23] Model Context Protocol: Specification 2024-11-05: Transports. https://modelcontextprotocol.io/specification/2024-11-05/basic/transports. Accessed 23 October 2026 (2024)

[24] Islam, M.A., Ali, M.E., Parvez, M.R.: MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. https://arxiv.org/abs/2405.11403 (2024)

[25] Zhang, Y., Ruan, H., Fan, Z., Roychoudhury, A.: AutoCodeRover: Autonomous Program Improvement. https://arxiv.org/abs/2404.05427 (2024)

[26] Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., Schmidhuber, J.: MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. https://arxiv.org/abs/2308.00352 (2023)

[27] Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., Sun, M.: ChatDev: Communicative Agents for Software Development. https://arxiv.org/abs/2307.07924 (2023)

[28] Huang, D., Zhang, J.M., Luck, M., Bu, Q., Qing, Y., Cui, H.: AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. https://arxiv.org/abs/2312.13010 (2023)

[29] Bouzenia, I., Devanbu, P., Pradel, M.: RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. https://arxiv.org/abs/2403.17134 (2024)

[30] Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., Ji, H.: Executable Code Actions Elicit Better LLM Agents. https://arxiv.org/abs/2402.01030 (2024)

[31] Phan, H.N., Nguyen, T.N., Nguyen, P.X., Bui, N.D.Q.: HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. https://arxiv.org/abs/2409.16299 (2024)

[32] Nguyen, M.H., Chau, T.P., Nguyen, P.X., Bui, N.D.Q.: AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology. https://arxiv.org/abs/2406.11912 (2024)

[33] Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A.: Experimentation in Software Engineering. Springer, Berlin, Heidelberg (2012)